

DOCUMENTAZIONE PROGETTUALE



Developed by *TEAM BRAVO*

Manuele Longo



<https://github.com/mlongo03>



<https://www.linkedin.com/in/manuele-longo-154979243/>

Flaviano Carlucci



<https://github.com/fcarlucc>



<https://www.linkedin.com/in/flaviano-carlucci-1aa816298/>

Introduzione

Questa documentazione fornisce una panoramica dettagliata dell'applicazione web, comprese l'architettura, le tecnologie e vari aspetti dello sviluppo e della distribuzione. L'obiettivo è fornire una chiara comprensione delle decisioni di design, dei dettagli di implementazione e di come interagiscono i vari componenti.

Panoramica dell'Architettura

Architettura Generale

L'architettura dell'applicazione è progettata per essere modulare e scalabile, utilizzando un'architettura a microservizi con frontend Angular e backend Spring Boot. La comunicazione tra i componenti avviene tramite API RESTful, e i servizi sono containerizzati utilizzando Docker. Questo approccio garantisce flessibilità, manutenibilità e scalabilità dell'applicazione.

Diagramma dell'Architettura

(Includere un diagramma che mostri l'interazione tra i componenti: frontend, API Gateway, microservizi e database. Indica anche la comunicazione tra i contenitori Docker.)

Componenti dell'Architettura

1. Frontend (Angular)

- **Descrizione:** Il frontend dell'applicazione è sviluppato utilizzando Angular, un framework per la costruzione di applicazioni web single-page (SPA). Angular gestisce l'interfaccia utente e la logica di presentazione, interagendo con il backend attraverso chiamate API REST.
- **Indirizzo IP:** Il frontend è servito su un indirizzo IP distinto dal backend, e comunica con il backend esclusivamente tramite le API esposte dal backend.

2. Backend (Spring Boot)

- **Descrizione:** Il backend è composto da una serie di microservizi sviluppati con Spring Boot. Ogni microservizio gestisce una specifica parte della logica di business e interagisce con un proprio database PostgreSQL. I microservizi sono progettati per essere indipendenti e scalabili, permettendo l'evoluzione e la manutenzione isolata di ciascun servizio.
- **Indirizzo IP:** I microservizi operano su indirizzi IP distinti e comunicano tra di loro all'interno dell'ambiente Docker. Ogni microservizio è esposto solo all'interno del cluster Docker e non direttamente alla rete esterna.

3. API Gateway

- **Descrizione:** L'API Gateway è un servizio centrale che funge da punto di ingresso per tutte le chiamate API esterne. Tutte le richieste provenienti dal frontend passano attraverso l'API Gateway, che gestisce la sicurezza, l'autenticazione e l'autorizzazione. L'API Gateway instrada le richieste ai

microservizi appropriati e può anche eseguire operazioni di load balancing e caching.

- **Porta Esposta:** Solo l'API Gateway è esposto su una porta della macchina host, permettendo così di gestire e filtrare l'accesso esterno all'intero sistema.

4. Microservizi

- **Descrizione:** Ogni microservizio è un'unità indipendente che gestisce una parte specifica dell'applicazione. I microservizi possono avere diverse responsabilità, come gestione degli utenti, elaborazione degli ordini, o altre funzionalità dell'applicazione.
- **Database:** Ogni microservizio è associato a un database PostgreSQL separato, garantendo l'isolamento dei dati e facilitando la scalabilità e la manutenzione dei dati specifici per ciascun servizio.

5. Containerizzazione (Docker)

- **Descrizione:** Tutti i componenti dell'applicazione, inclusi il frontend, i microservizi e i database, sono containerizzati utilizzando Docker. Ogni componente opera all'interno di un contenitore isolato, garantendo che le dipendenze e l'ambiente di esecuzione siano consistenti tra diversi ambienti (sviluppo, test, produzione).
- **Comunicazione tra Contenitori:** I contenitori comunicano tra di loro tramite una rete interna definita in Docker Compose. Solo l'API Gateway è esposto al mondo esterno, mentre gli altri microservizi e i database sono accessibili solo all'interno della rete Docker.

Motivazioni dell'Architettura

- **Flessibilità:** L'architettura a microservizi consente di aggiornare, scalare e distribuire i componenti in modo indipendente, migliorando la capacità di adattarsi ai cambiamenti e alle nuove esigenze.
- **Manutenibilità:** La separazione delle responsabilità tra i microservizi rende più facile il debugging, la manutenzione e l'evoluzione del codice.
- **Scalabilità:** La containerizzazione e l'isolamento dei microservizi permettono di scalare le risorse in base alle necessità, migliorando le prestazioni e la disponibilità dell'applicazione.
- **Gestione della Sicurezza:** L'API Gateway centralizza la gestione della sicurezza, semplificando l'autenticazione e l'autorizzazione e riducendo il rischio di esposizione diretta dei microservizi.

Tecnologie utilizzate

Frontend

Angular 18

- **Descrizione:** Angular è un framework per la costruzione di applicazioni web single-page (SPA). La versione 18 offre miglioramenti significativi in termini di prestazioni, modularità e supporto per le ultime caratteristiche di JavaScript.

- **Funzione:** Angular è utilizzato per sviluppare l'interfaccia utente dell'applicazione, gestendo la logica di presentazione e l'interazione con l'utente. Utilizza componenti, servizi e moduli per costruire un'esperienza utente dinamica e reattiva.
- **API Utilizzate:**
 - **Google Maps API:** Integra funzionalità di mappatura e geolocalizzazione nell'interfaccia utente, permettendo di visualizzare e interagire con mappe direttamente nel sito.
 - **ChatGPT API:** Utilizza l'API di ChatGPT per fornire funzionalità di chat automatizzata e assistenza all'utente in tempo reale.

Backend

Spring Boot 3.3.1

- **Descrizione:** Spring Boot è un framework per la creazione di applicazioni Java autonome e pronte per la produzione. La versione 3.3.1 include miglioramenti nella gestione delle configurazioni e nelle performance.
- **Funzione:** Ogni microservizio è sviluppato utilizzando Spring Boot, che gestisce la logica di business e le operazioni di backend. Spring Boot semplifica la creazione e la gestione di applicazioni basate su Spring, grazie alla sua configurazione automatica e ai suoi strumenti di sviluppo integrati.

Java 21

- **Descrizione:** Java 21 è una versione recente del linguaggio di programmazione Java, che include miglioramenti delle performance e nuove caratteristiche del linguaggio.
- **Funzione:** Utilizzato come linguaggio di programmazione per lo sviluppo dei microservizi. Java 21 fornisce una base robusta e sicura per la creazione di applicazioni scalabili e performanti.

PostgreSQL 17

- **Descrizione:** PostgreSQL è un sistema di gestione di database relazionali avanzato. La versione 17 offre nuove funzionalità e ottimizzazioni delle performance.
- **Funzione:** Ogni microservizio utilizza un database PostgreSQL separato per memorizzare e gestire i propri dati. PostgreSQL è scelto per la sua robustezza, supporto per query complesse e estendibilità.

Containerizzazione

Docker 26.1.4

- **Descrizione:** Docker è una piattaforma di containerizzazione che permette di creare, distribuire ed eseguire applicazioni all'interno di contenitori isolati.

- **Funzione:** Utilizzato per containerizzare i microservizi e i database, garantendo coerenza e isolamento tra i vari ambienti di sviluppo, test e produzione. La versione 26.1.4 include miglioramenti nella gestione dei contenitori e nelle performance.

Docker Compose 2.27.1

- **Descrizione:** Docker Compose è uno strumento che consente di definire e gestire multi-container Docker applications tramite file di configurazione YAML.
- **Funzione:** Utilizzato per definire e gestire l'ambiente di esecuzione dei contenitori, facilitando la configurazione e l'orchestrazione dei microservizi e dei loro database all'interno dell'ambiente Docker.

API di Google e Altri Servizi

API di Google per l'Autenticazione OAuth2

- **Descrizione:** Fornisce meccanismi di autenticazione sicura e autorizzazione per l'accesso alle risorse protette.
- **Funzione:** Utilizzata per gestire l'autenticazione degli utenti, permettendo agli utenti di accedere all'applicazione utilizzando i loro account Google. Questa integrazione facilita l'autenticazione e migliora la sicurezza.

API di Google per l'Invio di Email tramite Gmail

- **Descrizione:** Fornisce un'interfaccia per l'invio di email utilizzando il servizio Gmail.
- **Funzione:** Utilizzata per inviare notifiche e comunicazioni via email agli utenti dell'applicazione, sfruttando la robustezza e l'affidabilità del servizio Gmail.

Progettazione del sistema

Struttura del Sistema

L'applicazione è progettata utilizzando un'architettura a microservizi con un frontend Angular e un backend composto da più microservizi, tutti containerizzati con Docker. La progettazione è pensata per garantire flessibilità, scalabilità e manutenibilità del sistema.

Frontend (Angular)

Struttura e Componenti

Componenti

- L'interfaccia utente è costruita utilizzando componenti modulari di Angular.

- Ogni componente gestisce una parte specifica dell'interfaccia utente, come visualizzazioni delle pagine, moduli di input e widget interattivi.
- Alcuni esempi includono:
 - **HomeComponent**: Visualizza la pagina principale dell'applicazione.
 - **SignInComponent**: Gestisce il modulo di autenticazione degli utenti.
 - **ProfileComponent**: Visualizza e modifica i dettagli del profilo utente.

Servizi

- I servizi Angular centralizzano la logica di accesso ai dati e la gestione dello stato dell'applicazione.
- Alcuni esempi includono:
 - **AuthService**: Gestisce l'autenticazione e l'autorizzazione degli utenti.
 - **ApiService**: Fornisce metodi per comunicare con il backend con `api-calls`.
 - **ValidatorService**: Fornisce metodi personalizzati per validare gli input inseriti dall'utente

Routing

- Angular Router è utilizzato per gestire la navigazione tra le diverse viste dell'applicazione.
- La configurazione delle rotte è definita in moduli di routing dedicati.
- Utilizza guardie di rotte (**authGuard**, **authCoachGuard**, **authAdminGuard**) per proteggere le pagine che richiedono autenticazione.
- Esempi di rotte includono:
 - **/sign-in**: Rotta per il modulo di login.
 - **/profile**: Rotta per visualizzare e modificare il profilo utente.
 - **/admin/create-service**: Rotta per la creazione di nuovi servizi da parte degli amministratori.

Validazione

ValidatorService

- Il **ValidatorService** è un servizio centrale che fornisce una serie di funzioni di validazione personalizzate per garantire l'integrità e la correttezza dei dati inseriti nei moduli Angular.
- Questo servizio include varie funzioni di validazione specifiche per diversi tipi di input, assicurando che i dati rispettino i criteri predefiniti di formato e contenuto.

Direttive Personalizzate

- **ErrorMsgForm**: Una direttiva personalizzata per la gestione e visualizzazione degli errori di validazione dei moduli. Collabora strettamente con il **ValidatorService** per fornire un feedback immediato agli utenti.

- **ErrorMsgApi**: Una direttiva personalizzata per la gestione e visualizzazione degli errori delle chiamate api. Collabora strettamente con l' **ApiService** per fornire un feedback immediato agli utenti.

Interceptors

- Gli interceptor vengono utilizzati per gestire le richieste HTTP globali, includendo autenticazione, log e gestione degli errori.
 - **authInterceptor**: Aggiunge il token di autenticazione a tutte le richieste HTTP.
 - **loggingInterceptor**: Registra le richieste e le risposte HTTP per il debugging e l'analisi.
 - **headerInterceptor**: Aggiunge intestazioni personalizzate alle richieste HTTP.
 - **errorInterceptor**: Gestisce gli errori HTTP, mostrando messaggi appropriati all'utente.
 - **refreshTokenInterceptor**: Gestisce la logica di rinnovo del token di autenticazione quando scade.

Utilizzo di Interfacce

- Le interfacce vengono utilizzate per tipizzare le chiamate API e garantire la coerenza dei dati.

Integrazioni

Google Maps API

- Utilizzata per visualizzare mappe e gestire la geolocalizzazione all'interno dell'applicazione.
- I componenti Angular integrano l'API per visualizzare e interagire con le mappe.
- Esempio di utilizzo:
 - **NearMeComponent**: Visualizza una mappa con i servizi vicini all'utente.

ChatGPT API

- Integra funzionalità di chat automatizzata e assistenza all'utente.
- Utilizzata per offrire risposte in tempo reale e assistenza basata su AI.
- Fornisce un'interfaccia conversazionale per migliorare l'esperienza utente.

Backend (Spring Boot)

Microservizi

I seguenti microservizi compongono il back-end dell'applicazione:

1. Notification Service

- **Descrizione:** Questo servizio è dedicato all'invio di notifiche via email. Attualmente, gestisce l'invio di email singole tramite un account Gmail aziendale.
- **Funzioni:**
 - Creazione e invio di email di notifiche.
 - Configurazione dell'account di invio e gestione delle email in uscita.

2. OTP Service

- **Descrizione:** Gestisce i codici OTP (One-Time Password) utilizzati per l'autenticazione a due fattori e altre operazioni sicure.
- **Funzioni:**
 - Generazione e validazione di codici OTP.
 - Gestione della scadenza e dell'invalidamento dei codici.

3. Image Service

- **Descrizione:** Responsabile della gestione delle immagini all'interno dell'applicazione.
- **Funzioni:**
 - Validazione delle immagini caricate.
 - Gestione dell'upload e del download delle immagini.
 - Conservazione e recupero delle immagini nel sistema.

4. Comment Service

- **Descrizione:** Gestisce i commenti sui servizi offerti dall'azienda, che sono visualizzati sul sito web.
- **Funzioni:**
 - Validazione e creazione dei commenti.
 - Gestione delle interazioni con i commenti (es. modifica e cancellazione).

5. Like Service

- **Descrizione:** Gestisce il sistema di "like" sui servizi offerti.
- **Funzioni:**
 - Validazione e gestione della creazione e rimozione dei "like".
 - Aggregazione dei conteggi di "like" per i servizi.

6. Services Service

- **Descrizione:** Gestisce i servizi offerti dall'azienda.
- **Funzioni:**
 - Validazione, creazione e gestione dei dati sui servizi.
 - Invio e aggiornamento delle informazioni sui servizi attraverso chiamate API.

7. Shop Service

- **Descrizione:** Gestisce le operazioni di pagamento per i servizi.
- **Funzioni:**

- Gestione delle transazioni di pagamento attraverso una simulazione di un pagamento.
- 8. **Schedule Service**
 - **Descrizione:** Gestisce la pianificazione delle disponibilità dei welfare coach e la prenotazione di chiamate da parte degli utenti/dipendenti.
 - **Funzioni:**
 - Gestione delle disponibilità quotidiane dei welfare coach.
 - Gestione delle prenotazioni di chiamate e appuntamenti.
- 9. **User Service (Dipendenti)**
 - **Descrizione:** Gestisce gli account degli utenti/dipendenti dell'azienda.
 - **Funzioni:**
 - Validazione e creazione degli account utenti.
 - Gestione dei dati degli utenti e delle loro preferenze.
 - Fornitura delle informazioni degli utenti ai servizi che ne hanno bisogno.
- 10. **User Service (Welfare Coach)**
 - **Descrizione:** Gestisce gli account dei welfare coach.
 - **Funzioni:**
 - Validazione e creazione degli account dei welfare coach.
 - Gestione e aggiornamento dei dati dei coach.
 - Fornitura delle informazioni ai servizi e agli amministratori.
- 11. **API Gateway**
 - **Descrizione:** Funziona come il punto di ingresso centrale per tutte le richieste esterne e gestisce la comunicazione tra il frontend e i microservizi.
 - **Funzioni:**
 - Gestione della sicurezza e dell'autenticazione.
 - Instradamento delle richieste verso i microservizi appropriati tramite Spring Cloud Gateway.
 - Memorizzazione dei dati sensibili e gestione delle sessioni utente.
 - Implementazione di meccanismi di rate limiting per migliorare le prestazioni.

Comunicazione tra Microservizi

Network Interno del Cluster Docker:

- **Descrizione:** I microservizi comunicano tra di loro attraverso una rete interna definita all'interno del cluster Docker. Ogni servizio è esposto a porte e host noti all'interno di questo ambiente, garantendo che le comunicazioni avvengano in modo sicuro e isolato.
- **API RESTful:** La comunicazione tra i microservizi avviene tramite chiamate API RESTful. Ogni microservizio espone una serie di endpoint che possono essere utilizzati per effettuare richieste e scambiare dati.

Dettagli Tecnici

Configurazione dei Microservizi:

- **Contenitori Docker:** Ogni microservizio è eseguito in un contenitore Docker dedicato. La configurazione e la gestione dei contenitori sono gestite tramite Docker Compose.
- **Accesso ai Servizi:** I microservizi sono accessibili all'interno della rete Docker tramite nomi di servizio definiti nel file di configurazione di Docker Compose. Questo permette una comunicazione fluida e sicura tra i servizi.

Scalabilità e Manutenzione:

- **Scalabilità Orizzontale:** I microservizi possono essere scalati orizzontalmente, avviando più istanze di un servizio per gestire un aumento del carico.
- **Isolamento e Manutenzione:** La separazione dei microservizi consente di aggiornare e mantenere ciascun servizio in modo indipendente, riducendo i rischi di interruzioni e semplificando il deployment.

Sicurezza:

- **Autenticazione e Autorizzazione:** L'API Gateway gestisce l'autenticazione e l'autorizzazione per garantire che solo gli utenti autorizzati possano accedere ai servizi e ai dati.
- **Protezione dei Dati:** I dati sensibili sono gestiti e protetti tramite meccanismi di sicurezza implementati nell'API Gateway e nei microservizi.

Struttura del Database

La struttura del database è progettata per supportare i microservizi in modo efficiente, con ogni servizio che utilizza un proprio database PostgreSQL. Questa separazione garantisce l'isolamento dei dati e facilita la gestione e la scalabilità di ciascun servizio. Di seguito è riportata una panoramica della struttura dei database per ciascun microservizio, inclusi i dettagli sulle tabelle principali e le loro relazioni.

Database per l'OTP Service

Schema (otpservice):

```
otp
otp_id
expiry_time
otp_code
user_id
```

Database per l'Image Service

Schema (imageservice):

image
image_id
name
path
size
type

Database per il Comment Service

Schema (commentsservice):

comment
comment_id
commented_at
content
full_name
service_id
user_id

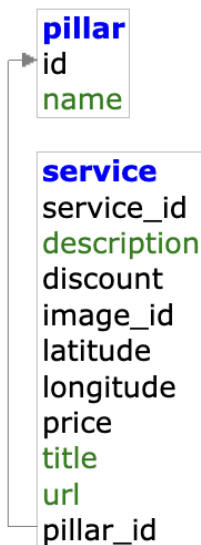
Database per il Like Service

Schema (likeservice):

like
like_id
service_id
user_id

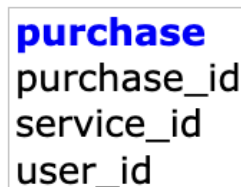
Database per il Services Service

Schema (servicesservice):



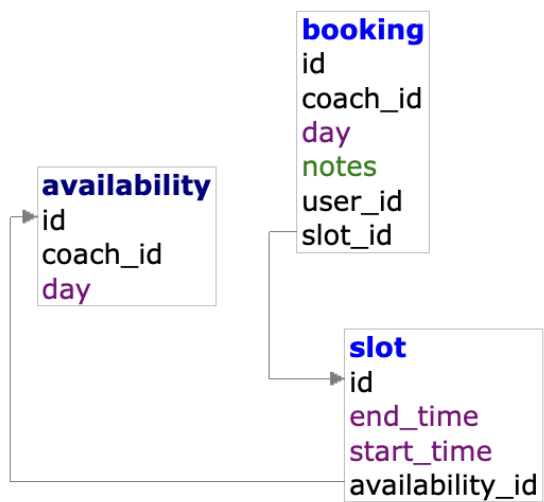
Database per il Shop Service

Schema (shopservice):



Database per il Schedule Service

Schema (scheduleservice):



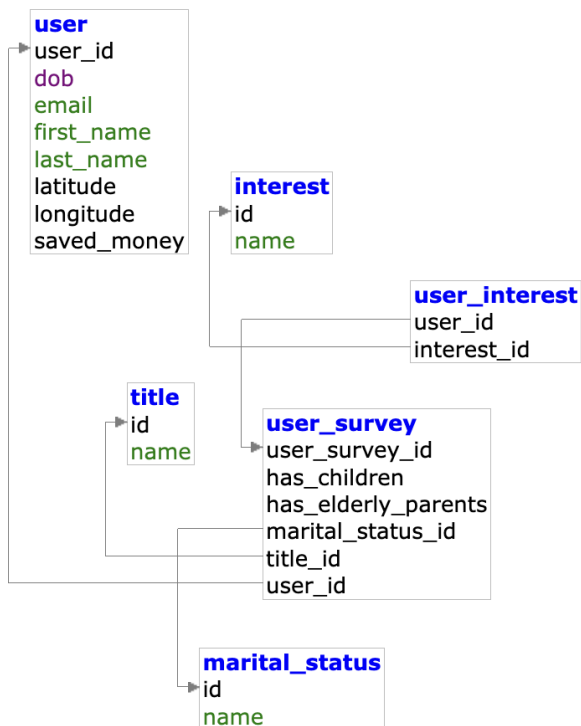
Database per il Coach Service

Schema (coachservice):



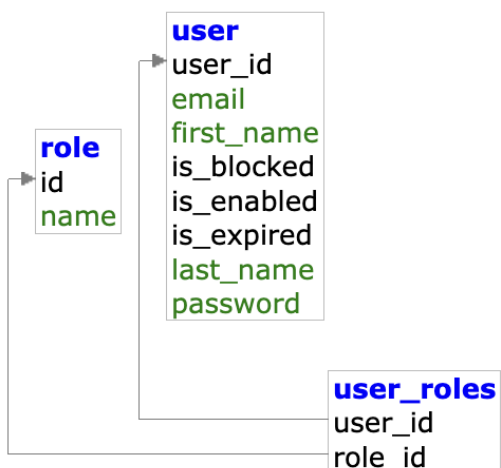
Database per il User Service

Schema (userservice):



Database per l'API Gateway

Schema (apigateway):



Sicurezza

La sicurezza è un aspetto fondamentale del nostro sistema, progettato e implementato per garantire la protezione dei dati degli utenti e la sicurezza delle operazioni. Questa sezione descrive le misure di sicurezza adottate per proteggere l'applicazione e i suoi dati.

Autenticazione e Registrazione

Validazione dei Dati

Descrizione: I dati inseriti nei form di registrazione e login sono validati sia lato frontend (Angular) che lato backend (Spring Boot) per garantire la correttezza e prevenire attacchi comuni come l'injection.

Implementazione:

- **Frontend:** Utilizzo di Angular Reactive Forms per la validazione immediata.
- **Backend:** Utilizzo di validatori di Spring Boot per una seconda fase di validazione dei dati.

Registrazione dell'Utente

Descrizione: Durante la registrazione, i dati dell'utente vengono salvati nel database, ma l'account rimane non verificato fino a quando non viene inserito un codice OTP ricevuto via email.

Processo:

1. L'utente compila il form di registrazione.
2. I dati vengono validati e, se corretti, l'utente viene creato nel database con stato "non verificato".
3. Viene generato un codice OTP e inviato all'utente via email.
4. L'utente inserisce il codice OTP per verificare l'account e abilitarlo all'uso.

Crittografia delle Password: Le password degli utenti sono criptate utilizzando bcrypt prima di essere memorizzate nel database.

Accesso e Autenticazione

Descrizione: Gli utenti possono effettuare il login tramite il sistema di login classico o tramite OAuth2 di Google.

Login Classico

Validazione: I dati di login vengono validati e verificati.

2FA: Se i dati sono corretti, l'utente deve inserire un codice OTP ricevuto via email per completare l'accesso.

Token JWT: Dopo la verifica, vengono generati due token JWT: un access token e un refresh token.

Stato 401: In caso di dati non corretti, viene restituito uno stato 401 Unauthorized.

OAuth2 con Google

1. Il frontend si reindirizza su una pagina specifica del backend che avvia l'autenticazione con Google tramite un URL fornito da Google.
2. Google valida il nostro client attraverso il client ID.
3. Dopo la validazione, Google genera un access token.
4. Utilizziamo questo access token per creare un nostro access token JWT e un refresh token.

Logout

Durante la procedura di logout, il refresh token viene rimosso dai cookie e l'access token viene rimosso dal session storage. Questo garantisce che l'utente non possa più accedere al sistema dopo aver effettuato l'uscita, contribuendo alla sicurezza complessiva dell'applicazione.

Token JWT

Descrizione dei Token JWT

Cos'è un Token JWT: JSON Web Token (JWT) è uno standard aperto per la creazione di token di accesso che permettono di verificare l'identità dell'utente e garantire la sicurezza delle comunicazioni.

Componenti del Token JWT:

- **Header:** Contiene informazioni sulla tipologia del token e l'algoritmo di crittografia utilizzato.
- **Payload:** Contiene le informazioni dell'utente (claims) e altri dati rilevanti.
- **Signature:** Utilizzata per verificare l'integrità del token.

Algoritmo di Crittazione HS256

I nostri token sono criptati utilizzando l'algoritmo HS256 (HMAC con SHA-256), uno degli algoritmi più affidabili e comunemente utilizzati per la crittografia di dati sensibili. La chiave di crittazione, utilizzata sia per la firma (signature) che per la decodifica dei token, è generata con estrema attenzione alla sicurezza, rappresentata in formato esadecimale (Hex).

Utilizzo dei Token JWT

Access Token

- Utilizzato per autenticare le richieste API.
- Ha una durata limitata di 3 minuti per motivi di sicurezza.
- Viene memorizzato nel session storage.
- Contiene informazioni non sensibili utili al frontend come fullname, email, id e ruolo.

Refresh Token

- Utilizzato per ottenere un nuovo access token senza dover fare nuovamente il login.
- Ha una durata di 24 ore.
- Viene memorizzato nei cookie con flag SameSite=strict, Path=/, Secure=true, HttpOnly=true per migliorare la sicurezza.

Zona di Immagazzinazione dei Token

I token vengono salvati in zone di memoria web differenti per vari motivi, tra cui flessibilità e sicurezza. L'access token viene salvato nel session storage, mentre il refresh token viene salvato nei cookie. Questo approccio previene il furto di entrambi i token e limita l'uso non autorizzato del sito.

Processo di Generazione

- Al momento del login, vengono generati sia l'access token che il refresh token.
- L'access token è incluso nelle richieste API per autenticare l'utente.
- Quando l'access token scade, il refresh token può essere utilizzato per ottenere un nuovo access token.

Processo di Refresh dell'Access Token

Descrizione: Quando una chiamata API arriva e il filtro JWT del backend nota che il token è valido ma scaduto, avvia il processo di refresh.

Passaggi:

1. Il backend rileva che l'access token è valido ma scaduto.
2. Si avvia in automatico il processo di refresh token.
3. Si verifica la presenza e la validità del refresh token.
4. Se il refresh token è valido, genera un nuovo access token.
5. Il nuovo access token viene inserito nell'header di risposta (Authorization: Bearer).
6. La richiesta continua ad essere elaborata normalmente.
7. Il frontend aggiorna il session storage con il nuovo access token.

Autorizzazione

Ruoli e Permessi

Descrizione: Implementiamo un sistema di autorizzazione basato su ruoli per controllare l'accesso alle risorse e alle funzionalità dell'applicazione.

Ruoli Principali:

- **Admin:** Accesso completo a tutte le funzionalità e ai dati.
- **Welfare Coach:** Accesso alle funzionalità di gestione delle disponibilità, delle chiamate e alla creazione di un nuovo servizio.
- **Dipendente:** Accesso alle funzionalità di visualizzazione e interazione con i servizi aziendali e con i welfare coach.

Gestione dei Permessi: Ogni endpoint API è protetto da controlli di autorizzazione che verificano i ruoli e i permessi dell'utente. Lato front-end, il ruolo presente nel token determina le funzionalità disponibili.

Protezione dei Dati

Crittografia dei Dati Sensibili

Descrizione: La sicurezza dei dati sensibili, come le password degli utenti e le chiavi API, è fondamentale per la nostra applicazione. Utilizziamo algoritmi di crittografia avanzati per proteggere queste informazioni.

Implementazione:

- **Password degli Utenti:** Le password vengono criptate utilizzando bcrypt. Questo algoritmo aggiunge un sale univoco a ogni password e applica un hashing ripetitivo, rendendo difficile per gli aggressori decifrare le password anche se ottengono l'accesso al database.
- **Chiavi API e Altri Dati Sensibili:** Le chiavi API e altri dati sensibili sono memorizzati nelle variabili ambientali dei container docker.

Gestione dei Codici OTP

Descrizione: I codici OTP (One-Time Password) sono utilizzati per verificare l'identità degli utenti in momenti chiave, come durante la registrazione o l'autenticazione a due fattori. La gestione sicura degli OTP è essenziale per prevenire accessi non autorizzati.

Implementazione:

- **Generazione dei Codici OTP:** I codici OTP sono generati in modo sicuro utilizzando un generatore di numeri casuali. Ogni codice è lungo 6 cifre e ha una durata di validità di 5 minuti.
- **Memorizzazione e Validità:** I codici OTP sono memorizzati nel database associati all'utente specifico. Ogni codice ha una scadenza di 5 minuti, dopo la quale non è più valido. Questo riduce il rischio di uso improprio dei codici OTP.
- **Validazione dei Codici OTP:** Durante la validazione, il codice OTP inserito dall'utente viene confrontato con quello memorizzato nel database. Se il codice è corretto e non è scaduto, viene accettato e subito dopo impostato a `null` per prevenire ulteriori utilizzi. Se il codice è scaduto, viene anch'esso impostato a `null` e la validazione fallisce.

Protezione delle Comunicazioni

Descrizione: Tutte le comunicazioni tra il frontend, il backend e i microservizi sono protette tramite HTTPS per prevenire intercettazioni e attacchi man-in-the-middle.

Certificati SSL: Utilizziamo certificati SSL per crittografare il traffico tra i client e i server.

Sicurezza dei Microservizi

Isolamento dei Contenitori

Descrizione: Ogni microservizio è eseguito in un contenitore Docker isolato, il che limita l'impatto di eventuali compromissioni e aumenta la sicurezza complessiva del sistema.

Network Interno

I microservizi comunicano tra loro attraverso un network interno del cluster Docker. Questo approccio riduce notevolmente l'esposizione agli attacchi esterni, poiché i microservizi non sono direttamente accessibili dall'esterno. L'unico servizio esposto esternamente è l'API Gateway.

API Gateway

L'API Gateway funge da unico punto di ingresso per tutte le richieste esterne. Essendo l'unico servizio esposto a una porta della host machine, garantisce che tutte le comunicazioni passino attraverso di esso. Questo consente di centralizzare e applicare tutte le misure di sicurezza, tra cui l'accesso, l'autenticazione, l'autorizzazione e la gestione dei dati sensibili degli utenti.

Variabili di Ambiente

Ogni container può accedere a delle variabili di ambiente che contengono dati sensibili, come chiavi API, credenziali di database e altri parametri di configurazione necessari per il funzionamento sicuro dei microservizi. L'uso delle variabili di ambiente permette di gestire questi dati in modo sicuro e centralizzato, senza doverli includere nel codice sorgente.

Vantaggi dell'Isolamento e del Network Interno:

- **Isolamento:** Ogni microservizio opera in un ambiente isolato, limitando il potenziale impatto di una compromissione.
- **Sicurezza della Comunicazione:** La comunicazione interna tra i microservizi avviene su una rete privata, non accessibile dall'esterno.
- **Protezione dei Dati Sensibili:** Utilizzando variabili di ambiente per gestire dati sensibili, si riduce il rischio di esposizione accidentale di tali informazioni.

Prevenzione delle SQL Injection

Descrizione

Per prevenire le SQL injection, implementiamo una serie di misure che garantiscono la sicurezza dei dati e delle operazioni.

Validazione dei Dati

Effettuiamo una valida e approfondita validazione dei dati inseriti sia lato frontend che lato backend:

- **Frontend:** Utilizziamo Angular Reactive Forms per validare i dati prima che vengano inviati al server.
- **Backend:** Utilizziamo validatori di Spring Boot per una seconda fase di validazione dei dati ricevuti.

Utilizzo di JPA

Adottiamo il framework JPA (Java Persistence API) per effettuare le query al database. Questo ci consente di scrivere query parametriche, che sono intrinsecamente sicure contro attacchi di SQL injection.

Rate Limiting

Descrizione

Per prevenire attacchi DDoS e brute force, abbiamo implementato un meccanismo di rate limiting nell'API Gateway. Questo filtro è stato aggiunto alla filter chain standard di Spring Security e controlla il numero di richieste provenienti da ciascun indirizzo IP al secondo.

Implementazione

Un filtro specifico è stato aggiunto alla filter chain di Spring Security per monitorare il numero di richieste effettuate da ogni indirizzo IP. Se un indirizzo IP supera il numero limite di richieste consentite, le richieste successive vengono bloccate.

Configurazione del Rate Limiter:

- **Filter Chain di Spring Security:** Utilizziamo un filtro custom nella catena di filtri di Spring Security per monitorare e limitare le richieste.
- **Limite di Richieste:** È impostato un limite massimo di richieste per ogni indirizzo IP. Per le chiamate di autenticazione, il limite è ancora più restrittivo per prevenire attacchi di brute force.

Protezione da XSS (Cross-Site Scripting)

Descrizione

Per prevenire attacchi XSS (Cross-Site Scripting), abbiamo adottato specifici header di sicurezza nelle nostre chiamate. Questi header proteggono l'applicazione limitando l'esecuzione di script non autorizzati e migliorando la sicurezza complessiva del sito.

Implementazione

Nella configurazione di sicurezza del nostro sito, abbiamo aggiunto gli header di protezione XSS e la politica di sicurezza dei contenuti (CSP) come segue:

- **Protezione XSS:** L'header `X-XSS-Protection` è impostato per bloccare le pagine se viene rilevato un potenziale attacco XSS. Utilizziamo

`XXssProtectionHeaderWriter.HeaderValue.ENABLED_MODE_BLOCK` per abilitare questa protezione e bloccare l'esecuzione di script dannosi.

- **Content Security Policy (CSP):** L'header `Content-Security-Policy` è configurato per limitare l'origine degli script che possono essere eseguiti sul sito. La direttiva `script-src 'self'` consente l'esecuzione solo degli script provenienti dal nostro dominio, prevenendo l'inclusione di script non autorizzati.

Protezione da Clickjacking

Descrizione

Per prevenire attacchi di tipo clickjacking, il nostro sito utilizza l'header `X-Frame-Options`. Questo header impedisce che le pagine del sito vengano caricate all'interno di un frame o iframe su un altro sito, garantendo così che gli utenti non vengano ingannati a cliccare su elementi nascosti.

Implementazione

Nella configurazione di sicurezza del nostro sito, abbiamo aggiunto l'header `X-Frame-Options` utilizzando `Customizer.withDefaults()` per applicare le impostazioni di default, che proteggono il sito da clickjacking:

Configurazione di Default:

- **DENY:** Impedisce che la pagina venga caricata in un frame o iframe su qualsiasi dominio.
- **SAMEORIGIN:** Permette il caricamento della pagina in un frame o iframe solo se il frame ha la stessa origine della pagina.

Utilizzando `Customizer.withDefaults()`, l'impostazione di default `SAMEORIGIN` viene applicata, garantendo che le pagine del nostro sito possano essere caricate in frame o iframe solo se l'origine del frame è la stessa del sito. Questo impedisce che domini esterni possano incapsulare le pagine del nostro sito, proteggendo così dagli attacchi di clickjacking.

Crittografia delle Comunicazioni con HTTPS

Il nostro sito utilizza HTTPS per garantire la sicurezza delle comunicazioni tra il frontend e il backend, proteggendo i dati degli utenti da intercettazioni e attacchi man-in-the-middle. Per implementare HTTPS, utilizziamo due chiavi SSL autofirmate: una per il frontend e una per il backend.

Chiavi SSL Autofirmate

- **Chiave per il Frontend:** Utilizzata per criptare il traffico tra i client (browser) e il server del frontend. Questa chiave garantisce che i dati scambiati tra gli utenti e l'applicazione Angular siano protetti e non possano essere letti o modificati da terzi.

- **Chiave per il Backend:** Utilizzata per criptare il traffico tra il frontend e il backend, così come tra i microservizi all'interno del cluster Docker. Questa chiave assicura che le comunicazioni tra i diversi componenti dell'architettura siano sicure e che i dati rimangano protetti durante il transito.

Vantaggi dell'Utilizzo di Due Chiavi Differenti

- **Separazione dei Ruoli:** Utilizzando due chiavi SSL diverse, possiamo separare chiaramente i ruoli e le responsabilità del frontend e del backend. Questo isolamento riduce il rischio di compromissione e semplifica la gestione delle chiavi.
- **Gestione Indipendente:** Le due chiavi possono essere gestite e aggiornate indipendentemente. In caso di necessità di rinnovo o aggiornamento di una chiave, l'altra chiave rimane operativa, garantendo la continuità del servizio.
- **Maggiore Sicurezza:** Utilizzare chiavi separate per frontend e backend aumenta la sicurezza complessiva. Se una chiave viene compromessa, l'accesso all'altra chiave rimane protetto, riducendo il rischio di un attacco riuscito che comprometta entrambe le aree.
- **Adattamento alle Policy di Sicurezza:** Diverse chiavi possono essere configurate con differenti politiche di sicurezza, in base alle esigenze specifiche dei componenti frontend e backend. Questo permette di applicare configurazioni più rigide e personalizzate per ciascuna parte dell'applicazione.

Configurazione della Filter Chain in Spring Security

Nel nostro API Gateway, la sicurezza è gestita attraverso una configurazione avanzata della filterchain di Spring Security. Questo processo è essenziale per garantire che tutte le richieste siano adeguatamente protette e che le risposte siano sicure. Ecco una panoramica ad alto livello della nostra configurazione:

1. **Filtri Personalizzati:** Abbiamo integrato filtri personalizzati per aggiungere ulteriori livelli di sicurezza. Ad esempio, un filtro di limitazione delle richieste viene applicato prima dei filtri di autenticazione predefiniti per prevenire abusi e limitare il numero di richieste che un singolo utente può effettuare in un determinato intervallo di tempo. Inoltre, un filtro per l'autenticazione basata su JWT è stato aggiunto per gestire l'autenticazione dei token e verificare la possibilità di rigenerare l'access token.
2. **Protezione dalle Richieste CSRF:** La protezione CSRF (Cross-Site Request Forgery) è disabilitata nella configurazione perché il nostro sistema è progettato per funzionare in modalità stateless, dove altre misure di sicurezza sono già in atto per gestire i rischi associati.
3. **Uso di HTTPS:** Tutte le comunicazioni tra il client e il server sono forzate ad utilizzare HTTPS, garantendo che i dati scambiati siano criptati e sicuri da intercettazioni.
4. **Gestione della Sessione:** La configurazione è impostata su una politica di sessione stateless, il che significa che non viene mantenuto alcun stato tra le richieste. Questo è particolarmente utile per applicazioni API che operano principalmente con token di autenticazione.
5. **Autorizzazione delle Richieste:** Le regole di autorizzazione sono configurate per determinare quali endpoint sono accessibili a quali ruoli. Alcuni endpoint sono

accessibili senza autenticazione, mentre altri richiedono ruoli specifici (come admin, coach o utenti) per l'accesso.

6. **Gestione delle Intestazioni di Sicurezza:** Sono state configurate intestazioni di sicurezza per proteggere contro attacchi comuni come XSS (Cross-Site Scripting) e per definire una politica di Content Security Policy (CSP) che limita le fonti da cui possono essere caricati script.
7. **Autenticazione:** L'autenticazione è gestita sia tramite OAuth2 che tramite form di login tradizionali. Per OAuth2, il sistema è configurato per reindirizzare gli utenti alla pagina di login di Google per l'autenticazione. Dopo un'accesso riuscito, i gestori specifici trattano i casi di successo e di fallimento dell'autenticazione.
8. **Logout:** Il processo di logout è gestito in modo che l'utente possa disconnettersi correttamente, con gestione specifica per la pulizia delle informazioni di autenticazione e il ripristino dello stato di sicurezza.
9. **Gestione delle Eccezioni:** La configurazione include gestori per le eccezioni che si occupano di rispondere a richieste non autorizzate e a tentativi di accesso negato, garantendo che gli utenti ricevano risposte appropriate e informative in caso di errori di autenticazione o autorizzazione.

Configurazione CORS

La configurazione CORS (Cross-Origin Resource Sharing) è fondamentale per permettere al nostro front-end di comunicare in modo sicuro con il backend, quando entrambi sono serviti da domini differenti. Ecco un riepilogo ad alto livello di come è impostata la nostra configurazione CORS:

1. **Permesso per Tutti gli Endpoint:**
 - Abbiamo configurato il server per accettare richieste CORS su tutti gli endpoint della nostra API. Questo significa che il frontend può effettuare chiamate a qualsiasi risorsa disponibile nel backend senza restrizioni sui percorsi specifici.
2. **Origini Consentite:**
 - La nostra configurazione specifica quali origini (domini) sono autorizzate a fare richieste al server. Questo include sia l'indirizzo IP del client con una porta specifica, sia senza la porta. Questa impostazione assicura che solo i client provenienti da indirizzi conosciuti e autorizzati possano interagire con l'API.
3. **Metodi HTTP Consentiti:**
 - Abbiamo abilitato i metodi HTTP necessari per le operazioni tipiche, come GET, POST, PUT, DELETE, e OPTIONS. Questo consente al frontend di eseguire tutte le operazioni principali (lettura, creazione, aggiornamento e cancellazione) e di gestire richieste complesse.
4. **Supporto per Credenziali:**
 - È stato abilitato il supporto per le credenziali, il che significa che il frontend può inviare cookie e altri dati di autenticazione con le richieste al backend. Questo è essenziale per gestire sessioni utente e autenticazione tramite token.

Documentazione API

La documentazione delle API del nostro progetto è essenziale per comprendere come interagire con i vari microservizi che compongono il sistema. Abbiamo predisposto una documentazione dettagliata utilizzando Javadoc, che genera siti web statici descrivendo tecnicamente il codice e fornendo una descrizione delle chiamate API nei pacchetti controller di ogni microservizio.

Come Visualizzare la Documentazione API:

1. Struttura della Documentazione:

- La documentazione è organizzata in modo che ogni microservizio abbia una propria sezione dedicata. Questo approccio modulare permette di trovare rapidamente le informazioni specifiche di ogni servizio.

2. Generazione dei Javadocs:

- I Javadocs sono generati automaticamente a partire dai commenti nel codice sorgente dei nostri microservizi (è possibile visualizzare i codici commentati nella cartella "Backend_docs/microservices_code_commented"). Questi commenti includono descrizioni delle classi, dei metodi e dei parametri, nonché informazioni specifiche sulle chiamate API.

3. Navigazione della Documentazione:

- Per visualizzare la documentazione, è necessario seguire questi passi:
 1. **Accedere alla cartella Backend_docs.**
 2. **Accedere alla cartella javadocs_microservices:** Questa cartella contiene la documentazione per tutti i microservizi.
 3. **Entrare nella cartella del microservizio desiderato:** Ogni microservizio ha una propria sottocartella all'interno di javadocs_microservices.
 4. **Aprire il file index.html:** All'interno della cartella specifica del microservizio, fare doppio clic sul file index.html. Questo aprirà il sito web statico generato da Javadoc nel browser predefinito.

4. Esplorazione dei Pacchetti:

- Una volta aperto il sito statico, è possibile navigare nei pacchetti del servizio. La documentazione delle API si trova principalmente nel pacchetto controller, dove sono descritte le varie chiamate API, i metodi disponibili, e i parametri necessari.

5. Descrizione Tecnica:

- La documentazione tecnica include informazioni dettagliate su ogni endpoint API, come le URI, i metodi HTTP utilizzati (GET, POST, PUT, DELETE), i parametri di richiesta e risposta, e i codici di stato HTTP ritornati. Questo è particolarmente utile per gli sviluppatori che devono integrare o mantenere il sistema.

